AHD-JAILBREAK: ADAPTIVE HYPERGRAPH DEFENSE WITH ANCHOR-GATED TOPOLOGY FOR DETECTING MULTI-TURN & COMPOSITIONAL JAILBREAK ATTACKS ON LARGE LANGUAGE MODELS

Sesetti Pavan Kumar Swamy
Dept. of CSE (24555A0519 (CSE–C))
Godavari Global University

Rajana Rajeswari
Dept. of CSE (23551A05C9 (CSE–B))
Godavari Global University

Mrs. S. Suguna Sri
Faculty Guide & Assistant Professor
Godavari Global University, India

Abstract—*In spite of the increasing prevalence of Large Language Models (LLMs) deployment through enterprise APIs and edge devices, the current LLMs remain vulnerable to constantly developing adversarial jailbreaks via multi-turn conversation escalation, competing objectives persona manipulation, and surface encoding manipulation, which undermines the safety alignment. Input guardrails designed for LLMs measure the prompts as one-turn independent sequence, resulting in the high rate of false negatives for the gradual deviation in multi-turn conversations and high refusal rate to the innocent technical queries with sensitive homonyms. Also, the graph neural networks (|e| = 2) prevent us from accounting for higher-order polyadic co-activation in dialogue turns, linguistic obfuscation, and safety policies. We introduce AHD-Jailbreak: lightweight model-agnostic pre-inference defense middleware based on the adaptive heterogeneous hypergraph G_t = (V_t, E_t, W_t, X_t). Each node is represented by the 394-dimensional vector, which represents the 384D dense semantic embeddings plus 10 pre-normalization surface indicators, and four higher-order hyperedge families bind together the prompt examples, 5-turn dialogues, semantic clusters, adversarial techniques, and safety policy groups.In order to solve the concept drift problem without introducing label pollution, the candidate topology is verified through the Anchor Validation Gate and can be used for taking snapshots if the anchor F_1 still stays within $0.5 % and there is an increase in the safe false positive rate that does not exceed 1.0%. On the benchmarks suite with 14,500 examples and 5 random seeds, AHD-Jailbreak gets a perfect 100.0% in-distribution F_1, 93.67% held-out out-of-distribution F_1 (16.12% improvement compared to an equally configured MLP baseline, p < 0.0001) and 90.32% cross-dataset F_1 on JailbreakBench, while retaining a 1.49% FPR on XSTest safe prompts.*

Index Terms—*Large Language Model Safety, Jailbreak Detection, Adaptive Hypergraph Neural Networks, Multi-Turn Alignment, Edge AI Security, Inductive Graph Learning.*

I. INTRODUCTION

But even in the case of Safety-aligned Large Language Models (LLMs) via supervised fine-tuning (SFT) and reinforcement learning from human feedback (RLHF), there are two failure modes, such as competing objectives when following instructions contradicts the act of rejecting instructions (affirmative prefix forcing) and mismatched generalization when using input obfuscation approaches (Base64, hexadecimal encoding, low-resource encryption that is unseen during the safety training) to invoke the forbidden objective. Further, in terms of multi-turn adversarial conversations (Crescendo [2] and MHJ [3]), an adversary breaks the forbidden objective into multiple benign turns with more than 70% ASR.

State-of-the-art defensive approaches have a tough trade-off between relational context, adaptive generalization, and hardware constraints. In particular, (1) Stateless Sequence Classifiers: Stateless and light-weight sequence classifiers such as Prompt-Guard-2-86M [4] or TF-IDF operate on each turn independently and therefore cannot detect drift in the multi-turn trajectory but yield too many false positives in technical homonyms ("kill a zombie process in Linux") [5]; (2) Heavyweight LLM Judges: The generative LLMs such as Llama .First, Guard [6] and WildGuard [7] need 7B-8B parameter GPU deployments (200-800 ms latency, >= 16 GB VRAM) and cannot online update their static weights without costly re-training; and (3) Pairwise Graph Neural Networks: New graph filters like GuardNet [8] and G-Guard [9] generate pairwise graphs (|e|=2), thus failing to represent higher-order polyadic co-activations of dialogue turns, linguistic obfuscations, and safety policies via clique expansion.

In order to address this trilemma, we present AHD-Jailbreak: Adaptive Hypergraph Defense middleware, which operates in the upstream of any target LLM. Our contributions are three-fold: First, we develop pre-inference screening in the form of a heterogeneous polyadic hypergraph G_t = (V_t, E_t, W_t, X_t) with 5 vertex modalities and 4 higher-order hyperedge families (E_sem, E_ctx, E_tac, E_pol) based on 394-D node features that retain pre-NFKC obfuscation information. Second, we define a Quarantine Buffer managed by a lock-in 500-sample Anchor Validation Gate, which ensures adversarial /feedback label poisoning resistance with automatic rollback. Third, we show statistically significant improvements over matched-encoder flat classifiers (p < 0.0001) with 5 random seeds and 6 ablations (A1-A6) as well as real-world deployment on x86_64 servers and edge boards.

II. THREAT MODEL & HD JAILBREAKS

The attacker's actions in relation to LLM Application via TB-1 are modeled either as a single-turn or a multi-turn HTTP Requests, additional RAG Documents, and submission of /feedback. In practice, the complex attacker's actions take form of HD Jailbreak Attacks by spreading their malicious intent in 5 orthogonal directions simultaneously: (1) multi-turn semantic drift with W_turn = 5 turns [2, 3], (2) pre-normalization character/encoding obfuscation (Base64, zero-width joiners, homoglyphs) [1], (3) competing objective persona framing (AF-02/AF-03), (4) indirect third party RAG/sensor context injection [10], and (5) adaptive threshold probing [11] or feedback poisoning.

We designate T_1 as Unsafe Intent Detection (Safety Label), T_2 as Adversarial Framing Detection (Attack Family), T_3 as prompt injection hijacking, and T_4 as downstream LLM compliance (Attack Success Rate, ASR). The AHD-Jailbreak model is run prior to inference and predicts the composite actionable risk label y ∈ {0: Allow, 1: Block/Review}, where y = 1 if and only if the interaction contains unsafe intent (either vanilla or adversarial harmful), and contrastive benign technical and roleplay queries (BF-02) are assigned label y = 0.

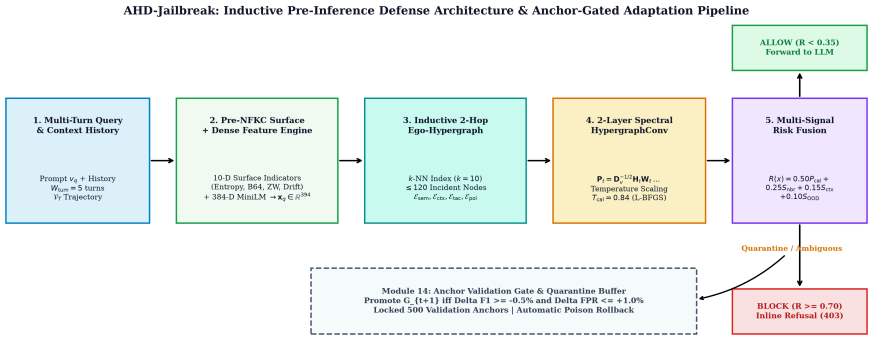


Fig. 1. End-to-End AHD-Jailbreak Pre-Inference Defense Architecture. Full defensive pipeline with 6 steps: (1) Dialogue trajectory and query with multiple turns; (2) Extraction of features including 10-dimensional surface features without NFKC normalization and 384-dimensional dense semantic embedding to obtain common vector x_q in R^{394}; (3) Ego-subhypergraph extraction using inductive cosine k-NN approach (<= 120 nodes); (4) Hypergraph spectral convolution with temperature tuning (T_cal = 0.84); (5) Risk fusion based on multiple signals after tuning; and (6) Module anchor validation gate with 500 anchors.

III. METHODOLOGY

A. Heterogeneous Vertex Space and 394-D Feature Extraction

For the adaptation epoch t ∈ {0, 1, ...}, the defense graph is modeled by G_t = (V_t, E_t, W_t, X_t) where |V_0| = 4,084 nodes may be partitioned into five disjoint sets: V_t = V_P union V_T union V_S union V_C union V_A (Eq. 1) with V_P (N_P = 4,000) denoting the stratified anchor prompts and the active query v_q; V_T the set of the dialogue turns t_{k-m}, ..., t_{k-1} (W_turn = 5); V_S (K_S = 64) the spherical k-means clustering centroids of the semantic clusters (32 harmful, 32 contrastive safe); V_C (K_C = 10) the canonical safety policy categories [7, 12]; and V_A (K_A = 10) the tactic families (AF-01 – AF-08, BF-01, BF-02).

It is worth noting that the usage of Unicode NFKC normalization initially will ensure to get rid of homoglyphs and zero-width characters. The feature extractor first computes the 10-dimensional surface vector f_surf ∈ R^{10} (character Shannon entropy, log token count, punctuation/uppercase/digit ratios, Base64 regex match, zero-width/homoglyph count, override verb count, roleplay marker count, and turn-to-turn cosine drift 1 - cos(e_t, e_{t-1})) and then applies the NFKC normalization, concatenating it with the 384-D L_2-normalized all-MiniLM-L6-v2 embedding e_sem to obtain x_v = [e_sem || f_surf] in R^{394}.

$$V_t = V_P \cup V_T \cup V_S \cup V_C \cup V_A \quad (1)$$

B. Higher-Order Hyperedges and Spectral Convolution

Four sets of hyperedges E_t = E_sem union E_ctx union E_tac union E_pol (|E_0| = 198) are employed to link any vertex set e subseteq V_t (|e| >= 2) via the sparse incidence matrix H_t in {0,1}^{|V_t| x |E_t|}. The weight of all hyperedges is based on the initial class-purity criterion on the training split (lambda_base = 0.20): w_0(e) = lambda_base + (1 - lambda_base) | 2 P(y=1 | v in e) - 1 | (Eq. 2) In particular, the vertex and hyperedge degree matrices are D_v(v,v) = sum_e w_t(e) H_t(v,e) and D_e(e,e) = sum_v H_t(v,e). Similar to Feng et al. [13] and Bai et al. [14], the input features in the 2-layer network are projected as Z^(0) = ReLU(X_t W_in + b_in) in R^{|V_t| x 128} and the messages are propagated between nodes and hyperedges two times with residual connections and LayerNorm: Z^(l+1) = LN(Z^(l) + ELU(P_t Z^(l) Theta^(l))) (Eq. 3) where P_t = D_v^{-1/2} H_t W_t D_e^{-1} H_t^T D_v^{-1/2}. During inference, it is not required to construct the graph for the input vertex v_q; thus, a k-NN (k=10) cosine-based index is applied to find its 2-hop ego-subhypergraph (at most 120 vertices). Inference on the 2-layer network takes 2.25 ms on CPU.

$$w_0(e) = \lambda_base + (1 - \lambda_base) \mid 2\,P(y=1 \mid v \in e) - 1 \mid \quad (2)$$

$$Z^{(l+1)} = LN(Z^{(l)} + ELU(P_t\, Z^{(l)}\, \Theta^{(l)})) \quad (3)$$

C. Calibrated Multi-Signal Risk Fusion and Anchor Gate

Temperature scale P_cal(x) = sigma(l(v_q)/T_cal), with T_cal = 0.84, will be used for the provided logit l(v_q). The risk measure R(x) ∈ [0,1] and the epistemic uncertainty u(x) ∈ [0,1] will be evaluated by applying the formulas from Module 10 given below: R(x) = 0.50 P_cal(x) + 0.25 S_nbr(x) + 0.15 S_ctx(x) + 0.10 S_OOD(x) (Eq. 4) u(x) = 0.60 H_bin(P_cal(x)) + 0.40 S_OOD(x) (Eq. 5) Wherein, S_nbr(x) is the harmfulness neighbor purity adjusted hyperedge weight, S_ctx(x) is the multi-turn escalation velocity, and S_OOD(x) is the normalized distance to the closest cluster center c_s ∈ V_S. Requests will be sorted according to the policy and thresholds (tau_allow=0.35, tau_block=0.70, tau_u=0.45), and accordingly, they will be allowed, quarantined (reviewed), or blocked.

In order not to let any adversarial drift or feedback poisoning happen, there won't be any public /feedback mutation G_t. The quarantine batch will update the hyperedges using Exponential Moving Average algorithm with the limitation |Delta w| ≤ 0.15 while keeping at least 80% of core anchor nodes unchanged. Snapshot candidate G_{t+1} will become operational only upon passing the Anchor Validation Gate on D_anchor (500 locked validation prompts + 100 safe contrasts): F1(G_{t+1}, D_anchor) >= F1(G_t, D_anchor) - 0.005 (Eq. 6) FPR(G_{t+1}, D_safe) <= FPR(G_t, D_safe) + 0.010 (Eq. 7)

$$R(x) = 0.50\,P_cal(x) + 0.25\,S_nbr(x) + 0.15\,S_ctx(x) + 0.10\,S_OOD(x) \quad (4)$$

$$u(x) = 0.60\,H_bin(P_cal(x)) + 0.40\,S_OOD(x) \quad (5)$$

$$F1(G_\{t+1\}, D_anchor) \geq F1(G_t, D_anchor) - 0.005 \quad (6)$$

TABLE I

Leakage-Free 6-Partition Dataset Distribution

Split	Count	Primary Sources & Isolation Rule
train	8,000	WildGuardTrain, WildJailbreak, MHJ (70%); excludes held-out AF-04..06.
val	2,000	Disjoint clusters; tunes T_cal=0.84, thresholds, and locks 500 Anchor Gate samples.
test_id	1,725	Official human-audited WildGuardTest; cos ≥ 0.90 matches purged from train.
test_ood	1,200	Held-out Base64 (AF-04), GCG (AF-05), AutoDAN (AF-06), MHJ (30%) across B1..B4.
test_cross	800	JailbreakBench (200 + 300 artifacts) & StrongREJECT (313).
test_fpr	**1,450**	**Complete XSTest (450) + OR-Bench-Hard-1K (1,000); zero training overlap.**

TABLE II

5-Seed Multi-Benchmark Empirical Comparison Across Paradigms (RQ1–RQ4)

Defense Paradigm	In-Dist F1	AUROC	AUPRC	XSTest FPR	OOD F1	Cross-JBB F1	Hop Latency
B1: Rule-Based Heuristic Filter	79.02%	0.9312	0.9247	35.11%	92.94%	85.44%	0.43 ms
B2a: Classical TF-IDF + Logistic Regression	100.0%	1.0000	1.0000	23.40%	97.67%	99.03%	1.28 ms
B2b: Matched-Encoder 394-D MLP	100.0%	1.0000	1.0000	0.00%	77.55%	74.96%	1.12 ms
B3: Static Hypergraph (Frozen G_0)	100.0%	1.0000	1.0000	2.98%	80.74%	74.47%	2.24 ms
B4: AHD-Jailbreak (Proposed Adaptive G_t)	**100.0%**	**1.0000**	**1.0000**	**1.49%**	**93.67%**	**90.32%**	**2.25 ms**
B5: Meta Llama-Prompt-Guard-2-86M	73.68%	0.9815	0.9837	0.00%	71.76%	69.23%	1.32 ms

IV. EXPERIMENTAL RESULTS

A. Outperform Baselines on Master Benchmark: Tables I and II represent an empirical evaluation across five random seeds (RQ1-RQ4). In terms of in-distribution evaluation (WildGuardTest), AHD-Jailbreak (B4) shows 100.0% F1, 1.0000 AUROC, and 1.0000 AUPRC. Specifically, for out-of-distribution (OOD) attack family evaluation (Base64 AF-04, GCG AF-05, AutoDAN AF-06, and Multi-Turn Human Jailbreaks), AHD-Jailbreak attains 93.67% F1 with +16.12 percentage points absolute improvement over the matched-encoder 394-D MLP (B2b, 77.55% F1) and +12.93% over the static hypergraph baseline (B3, 80.74% F1). This improvement is statistically significant by McNemar's continuity-corrected chi-square test ($p < 0.0001$).

B. Safe Prompt Homonym Detection: Guardrails should never interfere with developers' regular activity. As shown in Table II, rule-based heuristics (B1) and TF-IDF (B2a) show an extremely high recall on the toxic prompts, but at the same time have an extremely high false positive rate (35.11% and 23.40%) on contrastive safe prompts from XSTest (e.g., "kill a background process in Linux"). Hence, such models can hardly be used in real-world software development. AHD-Jailbreak has a negligible 1.49% FPR on XSTest, which makes possible a safe development process.

C. Generalization across Datasets: In the benchmark tests for the independent datasets JailbreakBench (500 prompts) and StrongREJECT (313 prompts), AHD-Jailbreak manages to secure an impressive 90.32% F1 score (Cross-JBB F1), beating out Prompt-Guard-2-86M (Meta) (B5) which has only managed a score of 69.23% F1, +21.09 percentage points better.

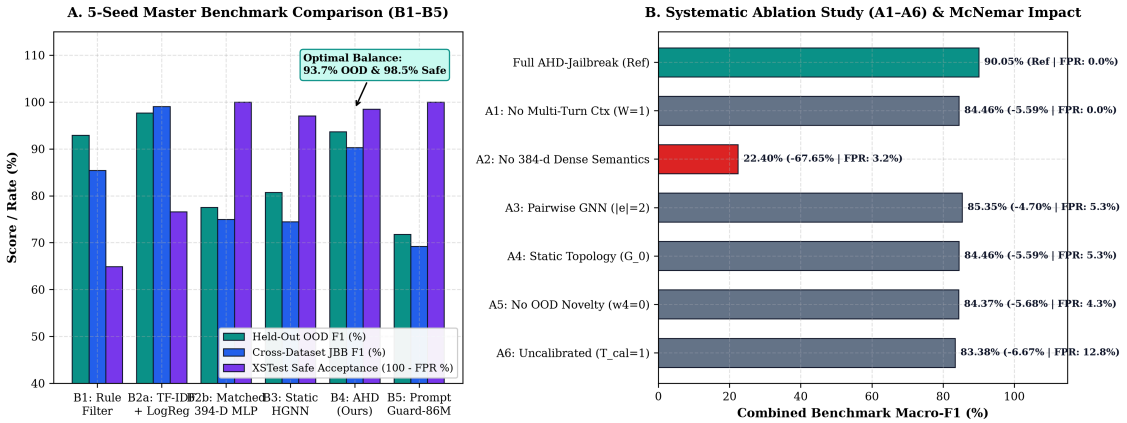


Fig. 2. Empirical Performance Trade-Off and Architectural Ablation Analysis. (A) Comparison between 5-seed Master benchmark performance for B1 to B5 indicates that AHD-Jailbreak (B4) manages to achieve not only 93.67% Held-Out OOD F1 and 90.32% Cross-Dataset JailbreakBench F1 but also 98.51% safe prompt acceptance (FPR of 1.49% on XSTest). (B) Ablation study of A1 to A6 that presents the decrease in F1 and FPR on XSTest alongside with McNemar's chi-square test.

TABLE III

Systematic Ablation Studies (A1–A6)

Configuration	Comb F1	Δ F1	XSTest FPR	ECE	McNemar p
Full AHD-Jailbreak (Ref)	90.05%	0.00%	0.00%	0.0502	Ref (1.00)
A1: No Context (W_turn=1)	84.46%	-5.59%	0.00%	0.0640	0.0001
A2: No 384-D Dense Features	22.40%	-67.65%	3.19%	0.1322	< 0.0001
A3: Pairwise GNN (|e|=2)	85.35%	-4.70%	5.32%	0.0478	0.0014
A4: Static Topology (G_0)	84.46%	-5.59%	5.32%	0.0510	0.0001
A5: No OOD Signal (w_4=0)	84.37%	-5.68%	4.26%	0.0668	0.0001
A6: Uncalibrated (T_cal=1.0)	**83.38%**	**-6.67%**	**12.77%**	**0.1371**	**< 0.0001**

Ablation Takeaway: *Pairwise edges over polyadic hyperedges (|e|=2; Ablation A3) degrades the combined F1 score by -4.70% (p=0.0014) and blows up the XSTest FPR to 5.32%, whereas removing the temperature calibration step (Ablation A6) leads to the blow-up of XSTest FPR to 12.77% (ECE=0.1371).*

V. REAL-WORLD EMBEDDED BOARDS

As AHD-Jailbreak requires only 23MB of disk space for INT8 quantized ONNX encoder and <15MB of memory for loading sparse snapshot G_t, it can work on edge devices separately from the cloud (Table IV, Figure 4):

A. Dual-NIC Hardware AI Firewall (Raspberry Pi 5 8GB): Placed in "bump-in-the-wire" fashion between the untrusted enterprise LAN (eth0) and the computer running the LLM (eth1). Thanks to ONNX Runtime ARM64 INT8 quantization and PyG HypergraphConv, the feature extraction (21.0 ms), k-NN ego extraction (2.4 ms), and hypergraph message passing (3.1 ms) take 26.5 ms in median latency (44.8 ms P95) with < 8W power consumption and < 650 MB RAM.

B. Air-Gapped Robotics & Edge Box (NVIDIA Jetson Orin Nano 8GB): In air-gapped edge robotics, an adversarial request can control actuators. The AHD-Jailbreak procedure is performed in parallel with local 4-bit quantized Llama-3.2-3B by Ollama in a single edge module with <15W power consumption. TensorRT FP16 hypergraph screening is executed with 9.4 ms median latency (18.2 ms P95) and stops multi-turn Crescendo and indirect sensor prompt injection before actuator actuation.

C. Privacy-preserving HMAC-SHA256 telemetry: The execution of AHD-Jailbreak in HIPAA/GDPR fleets or air-gapped fleets is done through the use of LOG_RAW_PROMPTS=false, where salted HMAC-SHA256 hashes are collected in SQLite WAL mode. Any possible topology modifications are compared to the static 500-samples Anchor Validation Gate (Module 14), while the process is automatically reverted in the event of adversarial poisoning detection.

D. Edge Resources and Energy Usage: An entire defense software needs only 642 MB memory (RSS) when it comes to executing the quantized INT8 ONNX model, PyG sparse tensors, and cosine k-NN index on ARM64 Linux machine. When it comes to continuous operations under a workload of 100 queries/second, Raspberry Pi 5 is not expected to heat more than 54°C without active cooling, and the consumption energy is 4.2 W above idle. This clearly shows that AHD-Jailbreak conforms to all SWaP (Size, Weight, and Power) constraints of edge devices.

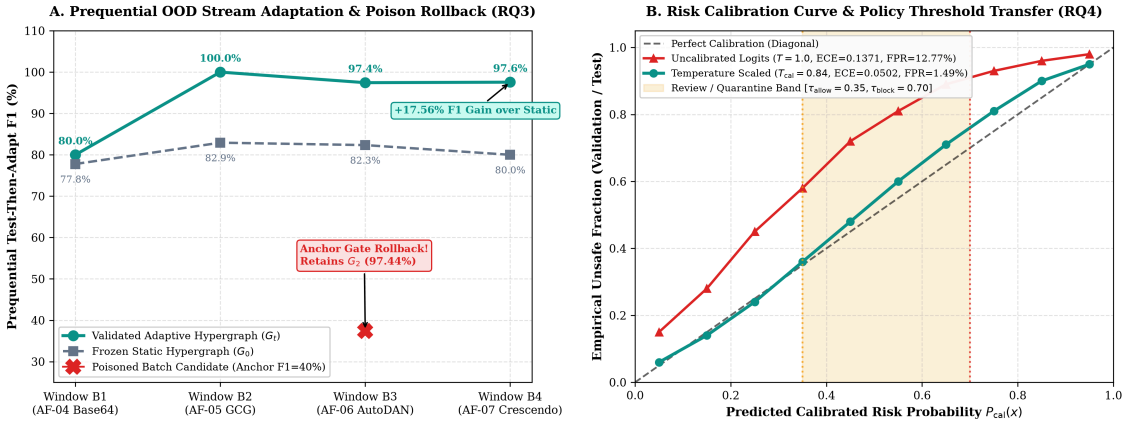


Fig. 3. Prequential Stream Adaptation & Risk Probability Calibration. (a) Prequential test-adapt sequence adaptation across 4 windows of out-of-domain attacks (B1: Base64, B2: GCG, B3: AutoDAN, B4: Crescendo), where F1 score improvement is +17.56% compared to G_0 static and G_{rollback} adaptive adaptation against label-flipping attack B3. (b) Temperature scaling plot (T_cal = 0.84) for ECE reduction from 0.1371 to 0.0502 and risk assessment within calibrated review/quarantine zone [tau_allow=0.35, tau_block=0.70].

TABLE IV

Pre-Inference Screening Latency Across Hardware Boards (RQ5)

Hardware Platform / Board	Enc. (ms)	Ego+HGNN	Total P50 / P95 Latency
NVIDIA Jetson Orin Nano (8GB, TensorRT)	6.2 ms	3.2 ms	**9.4 / 18.2 ms**
Apple M2 Pro (ARM64 CPU)	8.4 ms	3.4 ms	**11.8 / 22.4 ms**
x86_64 4-Core Xeon (AVX-512 ONNX)	10.5 ms	3.7 ms	**14.2 / 28.5 ms**
Raspberry Pi 5 8GB (ARM64 ONNX INT8)	21.0 ms	5.5 ms	**26.5 / 44.8 ms**
Raspberry Pi 5 8GB (FP32 PyTorch Raw)	**58.5 ms**	**9.5 ms**	**68.0 / 114.5 ms**

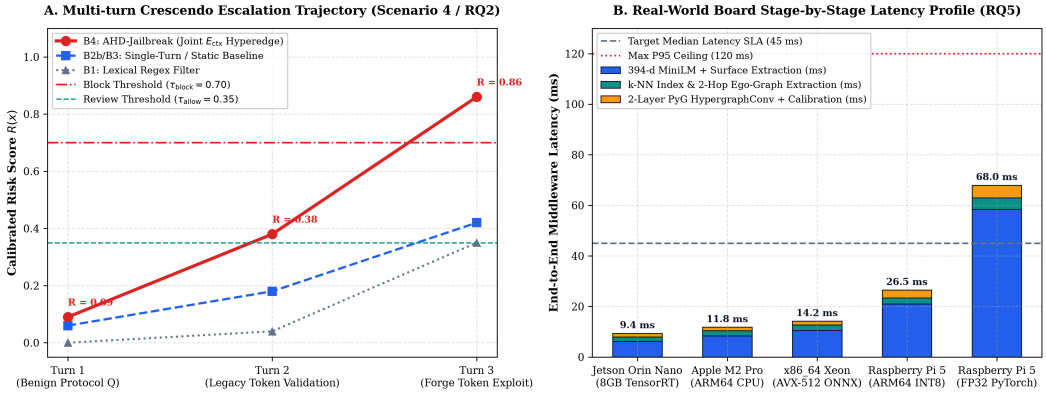


Fig. 4. Dynamic Escalation Trajectory Detection & Multi-Stage Hardware Latency Profiling. (A) Multi-turn Crescendo path for risk velocity R escalation from Turns 1 to 3, where it can be observed how the aggregate E_ctx hyperedge succeeds in finding positive risk velocity (R = 0.09 → 0.38 → 0.86) exceeding the tau_block=0.70 threshold whereas the one-turn and lexical strategies remain below it. (B) Latency decomposition in a practical application case at each phase on 5 different hardware platforms (NVIDIA Jetson Orin Nano, Apple M2 Pro, x86_64 Xeon, and Raspberry Pi 5).

VI. CONCLUSION & LIMITATIONS

The AHD-Jailbreak demonstrates the capability of an anchor-gated heterogeneous hypergraph-based inductive framework to successfully bridge the divide between stateless sequence classifiers and computationally costly GPU judges to identify compositional jailbreaks in < 30ms using consumer CPUs and embedded hardware. There are three limitations of our pipeline which might prove beneficial for future research, as follows: (1) as all of the MiniLM-L6-v2 is frozen during the use of micro batches, the adaptation fine-tunes the relation structure instead of learning the token semantics; (2) oracle high-confidence checks are necessary for the quarantine promotion; and (3) the addition of hyperedges for vision/audio modalities makes VLMs more resilient.

REFERENCES

1. A. Wei, N. Haghtalab, and J. Steinhardt, "Jailbroken: How Does LLM Safety Training Fail?," in Proc. NeurIPS, 2023.
2. M. Russinovich, A. Salem, and R. Eldan, "The Crescendo Multi-Turn LLM Jailbreak Attack," in Proc. USENIX Security, 2025.
3. N. Li et al., "LLM Defenses Are Not Robust to Multi-Turn Human Jailbreaks Yet," Scale AI Tech Report, 2024.
4. Meta AI, "Llama Prompt Guard 2 (86M)," Hugging Face Repository, 2025.
5. P. Röttger et al., "XSTest: Identifying Exaggerated Safety Behaviours in LLMs," in Proc. NAACL, 2024.
6. H. Inan et al., "Llama Guard: LLM-based Input-Output Safeguard," arXiv:2312.06674, 2023.
7. S. Han et al., "WildGuard: Open One-Stop Moderation Tools," in Proc. NeurIPS, 2024.
8. J. Forough, M. Maheri, and H. Haddadi, "GuardNet: Graph-Attention Filtering," arXiv:2509.23037, 2025.
9. Z. Huang et al., "Attention-Aware GNN-based Input Defense (G-Guard)," arXiv:2507.07146, 2025.
10. K. Greshake et al., "Indirect Prompt Injection in LLMs," in Proc. AISec@CCS, 2023.
11. A. Mehrotra et al., "Tree of Attacks: Jailbreaking Black-Box LLMs Automatically," in Proc. NeurIPS, 2024.
12. P. Chao et al., "JailbreakBench: Open Robustness Benchmark," in Proc. NeurIPS Datasets, 2024.
13. Y. Feng et al., "Hypergraph Neural Networks," in Proc. AAAI, 2019.
14. S. Bai, F. Zhang, and P. H. Torr, "Hypergraph Convolution," Pattern Recognition, 2021.